

Wydział Informatyki Politechniki Białostockiej Systemy Baz Danych	Data oddania: 19.06.2026 r.
Pracownia specjalistyczna nr 5 Zofia Stanisławska	Prowadzący: prof. dr hab. Piotr Hońko

Opis aplikacji bazodanowej

Projektowana baza danych jest dla aplikacji typu Tinder dla ryb.

Projekt realizowany przy pomocy Backendu .Net + SwaggerUI.

Używa trzech pakietów:

[BCrypt.Net](#)-Next - do hashowania i weryfikacji haseł

Oracle.ManagedDataAccess.Core - do komunikacji z bazą oracle

Swashbuckle.AspNetCore - do generowania SwaggerUI i dokumentacji API

Wymagania funkcjonalne aplikacji:

- Rejestracja i logowanie użytkowników
- Tworzenie i edycja profilu
- Wyszukiwanie profili dopasowanych pod użytkownika
- Wysyłanie zaproszeń i tworzenie konwersacji
- Panel Administratora do zarządzania użytkownikami, gatunkami oraz zainteresowaniami.

Wymagania nie funkcjonalne aplikacji:

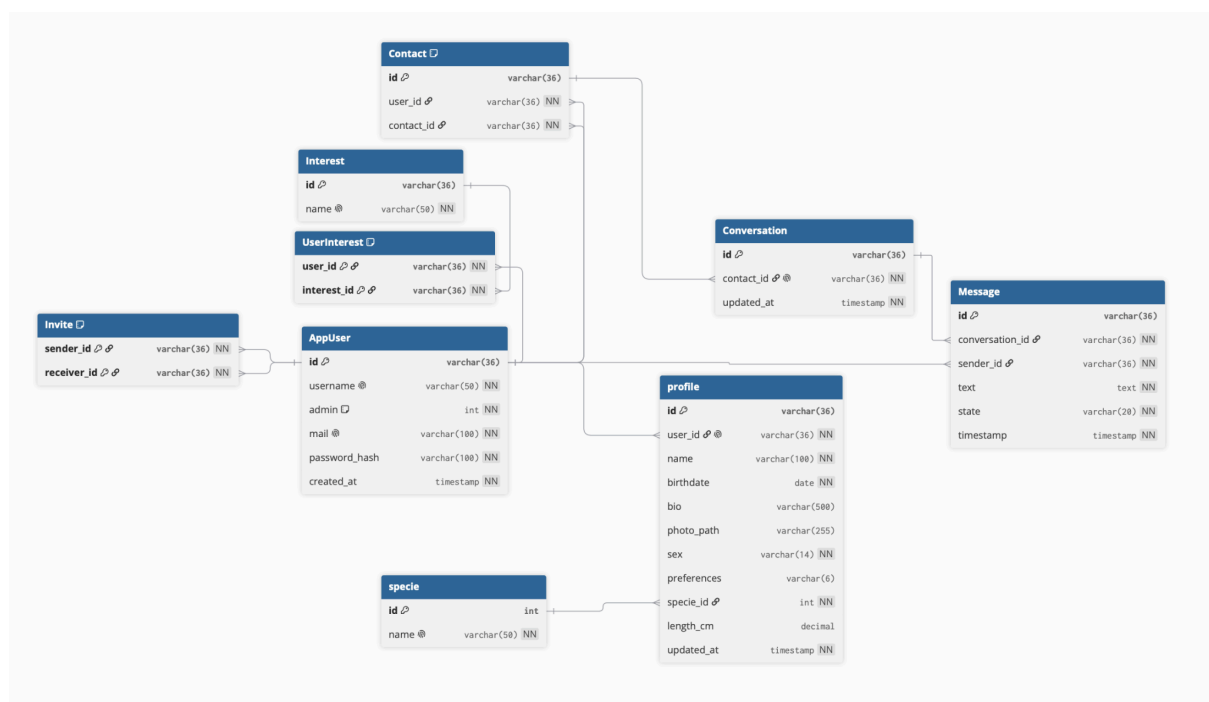
- Bezpieczeństwo przechowywania haseł
- Integralność danych przez Check
- Walidacja wieku użytkownika
- Dokumentacja API przez Swagger
- Modularność poprzez pakiety PL/SQL

1. Lista encji

- **AppUser** - Konta użytkowników – login, e-mail, hash hasła, rola administratora
- **Profile** - Profil użytkownika, cechy osobiste i preferencje
- **Specie** - Gatunki (specyficzne właściwości istotne dla rankingu/porównań ryb)
- **Contacts** - Lista kontaktów/friends (relacja użytkownik-użytkownik)
- **Conversation** - Rozmowy (kanały komunikacji powiązane z relacjami/kontaktami)
- **Invite** - Zaproszenia do kontaktów (relacja użytkownik-użytkownik)

- **Messages** - Wiadomości przesyłane w ramach rozmów
- **Interest** - Lista dostępnych zainteresowań do wybrania
- **UserInterest** - Tabela (Wiele do wielu) co łączy Użytkownika z wieloma zainteresowaniami

Wygląd Bazy danych:



2. Rozmiar początkowy i zakładany przyrost danych

Początkowa liczba użytkowników: 100

Początkowa liczba profili: 100 (tyle samo co użytkowników zawsze)

Początkowa liczba gatunków (specie): 20-50
Początkowa liczba kontaktów (contacts): 400
Początkowa liczba rozmów: 400 (tyle samo co znajomości)
Początkowa liczba wiadomości: 2000
Początkowa liczba zainteresowań: 100
Początkowa liczba zainteresowań użytkowników: 500 (każdy użytkownik po ok. 5 zainteresowań w profilu)

Przyrost danych:

Użytkownicy i Profile: +20/miesiąc
Wiadomości: +500/miesiąc
Kontakty i rozmowy : +50/miesiąc

3. Identyfikacje tabel o największym obciążeniu (przeszukiwanie / ilość danych)

Tabele o największej ilości danych to będą messages, invites. A najczęściej przeszukiwane profile.

4. Mechanizm historii zmian i elementów

Zalecane robienie backupów na 3 oddzielnych komputerach plus jednym dysku przenośnym.

5. Role i uprawnienia / rodzaje użytkowników

- bazowy zalogowany użytkownik - ma dostęp do wyszukiwarki profili ryb może wysyłać wiadomości do swoich wybranych co i on wybrał, edytować tworzyć oraz edytować swój profil.
- admin - uprawnienia bazowego zalogowanego użytkownika + usuwanie dodawanie zainteresowań i gatunków, oraz usuwanie użytkowników.

6. Dodatkowe metody zabezpieczeń danych wrażliwych

Zamiast normalnego indeksowania profili użytkowników przez integer jest użyte UUID.

Hasła są Hashowane.

Zastosowanie kluczy obcych by zapobiec osieroceniu danych.

7. Widoki, triggerzy, pakiety z funkcjami i procedurami.

Zastosowano w projekcie bazy danych pakietowanie by można było bardziej przejrzysto zarządzać kodem.

PKG_AUTH - dla zarządzania użytkownikiem (rejestracja, logowanie, wylogowywanie).

-procedura register_user - rejestracja nowego użytkownika

-procedura get_user_by_name - pobiera dane użytkownika na podstawie loginu

PKG_PROFILE – zarządzanie profilami (tworzy, modyfikuje, pobiera dane profili)

-procedura create_profile - tworzy użytkownika

-procedura update_profile - aktualizuje profil

-procedura get_interests - pobiera listę zainteresowań użytkownika

-procedura add_interest - dodaje zainteresowania do listy zainteresowań użytkownika

- procedura remove_interest - usuwa zainteresowania użytkownika

- funkcja get_age() – oblicza wiek użytkownika na podstawie daty urodzenia

PKG_DISCOVERY – wyszukiwanie profili

- funkcja are_compatible() - sprawdza czy użytkownicy pasują do siebie bazując na gatunku, płci i preferencji

-get_all_profiles - pobiera wszystkie profile innych użytkowników możliwe jest też filtrowanie po płci

-procedura get_compatible_profiles - pobiera profile pasujące do użytkownika

PKG_SOCIAL – kontakty i zaproszenia

- procedura send_invite - wysyła zaproszenie do innego użytkownika

-procedura get_contacts - pobiera listę kontaktów użytkownika

PKG_MESSAGING – obsługa wiadomości

- funkcja count_unread() - liczy nieprzeczytane wiadomości

-procedura send_message - dodaje nową wiadomość do konwersacji

-procedura mark_as_read - oznacza wiadomości jako przeczytane

-procedura get_messages - pobiera historię wiadomości użytkownika (od najstarszej do najnowszej)

PKG_ADMIN – funkcje administracyjne (dodawanie usuwanie gatunków i zainteresowań, usuwanie użytkowników)

-procedura delete_user - usuwa użytkownika

-procedura add_specie - dodaje nowy gatunek

-procedura add_interest - dodaje nowe zainteresowanie

-procedura get_all_specie - wyświetla wszystkie gatunki

-procedura get_all_interests - pobiera wszystkie zainteresowania

Użyto triggerów by zautomatyzować procesy oraz zapewnić integralność danych przez co baza danych jest niezależna od aplikacji.

trg_invite_mutual - Uruchamiany po dodaniu zaproszenia. Sprawdza w tabeli czy użytkownik A zaprosił użytkownika B i w przeciwną stronę. Gdy jest to spełnione tworzy od razu kontakt dla nich oraz konwersacje by mogli wysyłać do siebie wiadomości.

trg_profile_check_age - Uruchamiany przed dodaniem lub aktualizacją profilu. Sprawdza wiek użytkownika czy ma już 18 lat. Jest on stworzony aby zapewnić integralność danych, w przypadku gdyby z strony aplikacji przeszłoby dodanie nieletniego użytkownika.

Nie użyto do projektu bazy danych widoków ponieważ większość zapytań to procedury zwracające kursory ponieważ potrzebują używać parametrów. Dodatkowo wszystko znajduje się w pakietach więc tworzenie widoków byłoby niepotrzebnym powieleniem kodu.

Przykładowe zapytania i ich plany by zoptymalizować bazę danych:

Zanim przesłałam do zrobienia tych planów wypełniłam tabele profile(1000 wierszy),Specie(5),contact(200),conversation(200).

Procedura get_profile z PKG_PROFILE.

Id	Operation	Name	Rows	Bytes	Cost (%CPU)
0	SELECT STATEMENT		1	223	4 (0)
1	NESTED LOOPS		1	223	4 (0)
2	NESTED LOOPS		1	212	3 (0)
3	TABLE ACCESS BY INDEX ROWID	PROFILE	1	149	2 (0)
* 4	INDEX UNIQUE SCAN	SYS_C008641	1		1 (0)
5	TABLE ACCESS BY INDEX ROWID	APPUSER	1	63	1 (0)
* 6	INDEX UNIQUE SCAN	SYS_C008629	1		0 (0)
7	TABLE ACCESS BY INDEX ROWID	SPECIE	1	11	1 (0)
* 8	INDEX UNIQUE SCAN	SYS_C008605	1		0 (0)
Predicate Information (identified by operation id):					

4 - access("P"."USER_ID"='549FEFBDDCE0B27E063020011AC9EF2')					
6 - access("U"."ID"='549FEFBDDCE0B27E063020011AC9EF2')					
8 - access("S"."ID"="P"."SPECIE_ID")					

Wszystko wygląda dobrze, nie ma potrzeby tu nic robić.

Procedura get_all_profiles z Pkg_discovery - Filtrowanie ustawione na "female" by znaleźć wszystkie kobiety.

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	
0	SELECT STATEMENT		1	144	10	(0)
* 1	HASH JOIN		1	144	10	(0)
2	NESTED LOOPS		1	144	10	(0)
3	NESTED LOOPS		1	144	10	(0)
4	STATISTICS COLLECTOR					
* 5	TABLE ACCESS FULL	PROFILE	1	133	9	(0)
* 6	INDEX UNIQUE SCAN	SYS_C008605	1		0	(0)
7	TABLE ACCESS BY INDEX ROWID	SPECIE	1	11	1	(0)
8	TABLE ACCESS FULL	SPECIE	1	11	1	(0)

Predicate Information (identified by operation id):						

1 - access("S"."ID"="P"."SPECIE_ID")						
5 - filter("P"."SEX"='female' AND						
"P"."USER_ID"<>'549FEFBDDECE0B27E063020011AC9EF2')						
6 - access("S"."ID"="P"."SPECIE_ID")						

Tutaj widać, że jest Table Access Full do tabeli Profile to znaczy, że zapytanie skanuje całą tabelę co robi nam wąskie gardło.

Zrobiony więc został index:

CREATE INDEX idx_profile_sex_user ON Profile(sex, user_id);

bo plan w punkcie 5 podpowiedział że to tu jest możliwe obciążenie.

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	
0	SELECT STATEMENT		1	144	4 (0)	
* 1	HASH JOIN		1	144	4 (0)	
2	NESTED LOOPS		1	144	4 (0)	
3	NESTED LOOPS		1	144	4 (0)	
4	STATISTICS COLLECTOR					
5	TABLE ACCESS BY INDEX ROWID BATCHED	PROFILE	1	133	3 (0)	
* 6	INDEX RANGE SCAN	IDX_PROFILE_SEX_USER	1		2 (0)	
* 7	INDEX UNIQUE SCAN	SYS_C008605	1		0 (0)	
8	TABLE ACCESS BY INDEX ROWID	SPECIE	1	11	1 (0)	
9	TABLE ACCESS FULL	SPECIE	1	11	1 (0)	

Predicate Information (identified by operation id):

1 - access("S"."ID"="P","SPECIE_ID")
6 - access("P"."SEX"='female')
filter("P"."USER_ID"<>'549FEFBDDCE0B27E063020011AC9EF2')
7 - access("S"."ID"="P","SPECIE_ID")

Zadziałało, teraz szuka na indexie i Cost jest o ⅓ mniejszy.

Plan dla zapytania get_contacts

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	
0	SELECT STATEMENT		1	255	6 (17)	
1	SORT ORDER BY		1	255	6 (17)	
* 2	HASH JOIN		1	255	5 (0)	
3	NESTED LOOPS		1	255	5 (0)	
4	NESTED LOOPS		1	255	5 (0)	
5	STATISTICS COLLECTOR					
* 6	HASH JOIN		1	176	4 (0)	
7	NESTED LOOPS		1	176	4 (0)	
8	STATISTICS COLLECTOR					
* 9	TABLE ACCESS FULL	CONTACT	1	99	3 (0)	
10	TABLE ACCESS BY INDEX ROWID	CONVERSATION	1	77	1 (0)	
* 11	INDEX UNIQUE SCAN	SYS_C008662	1		0 (0)	
12	TABLE ACCESS FULL	CONVERSATION	1	77	1 (0)	
* 13	INDEX UNIQUE SCAN	SYS_C008641	1		0 (0)	
14	TABLE ACCESS BY INDEX ROWID	PROFILE	1	79	1 (0)	
15	TABLE ACCESS FULL	PROFILE	1	79	1 (0)	

Predicate Information (identified by operation id):

Nie jest źle ale może być lepiej, tutaj Table access na Tabeli Contact nie jest aż taki złośliwy (3) ale można i tak go poprawić, na tabeli Conversation zostawimy go full access bo (1) to całe nic.

3	Id	Operation	Name	Rows	Bytes	Cost (%CPU)
4						
5	0	SELECT STATEMENT		1	255	5 (20)
6	1	SORT ORDER BY		1	255	5 (20)
7	* 2	HASH JOIN		1	255	4 (0)
8	3	NESTED LOOPS		1	255	4 (0)
9	4	NESTED LOOPS		1	255	4 (0)
10	5	STATISTICS COLLECTOR				
11	* 6	HASH JOIN		1	176	3 (0)
12	7	NESTED LOOPS		1	176	3 (0)
13	8	STATISTICS COLLECTOR				
14	9	TABLE ACCESS BY INDEX ROWID BATCHED	CONTACT	1	99	2 (0)
15	10	BITMAP CONVERSION TO ROWIDS				
16	11	BITMAP OR				
17	12	BITMAP CONVERSION FROM ROWIDS				
18	* 13	INDEX RANGE SCAN	IDX_CONTACT_USER			1 (0)
19	14	BITMAP CONVERSION FROM ROWIDS				
20	* 15	INDEX RANGE SCAN	IDX_CONTACT_CONTACT			1 (0)
21	16	TABLE ACCESS BY INDEX ROWID	CONVERSATION	1	77	1 (0)
22	* 17	INDEX UNIQUE SCAN	SYS_C008662	1		0 (0)
23	18	TABLE ACCESS FULL	CONVERSATION	1	77	1 (0)
24	* 19	INDEX UNIQUE SCAN	SYS_C008641	1		0 (0)
25	20	TABLE ACCESS BY INDEX ROWID	PROFILE	1	79	1 (0)
26	21	TABLE ACCESS FULL	PROFILE	1	79	1 (0)
27						

Poprawiło się.